

KEYLOGGER DETECTION SYSTEM: A DEFENSIVE RULE-BASED PROCESS ANALYSIS APPLICATION

Abdulmutin

Kastamonu University, Department of Computer Engineering, 2nd Year
Object-Oriented Programming Course Project Report

Abstract- This report presents the Keylogger Detection System, a defensive student project developed with ASP.NET Core MVC, Entity Framework Core, and PostgreSQL. The application does not create malware, does not collect keystrokes, and does not monitor live operating system processes. Instead, it analyzes manually entered process/log-like records using detection rules stored in the database. Triggered rules are converted into a risk score, and the final result is classified as low, medium, or high risk. The project demonstrates object-oriented programming concepts through entity classes, service interfaces, service implementations, controllers, and view models. It also includes authentication, role-based authorization, analysis history, reports, detection rule management, user management, and a dedicated ethical usage page.

Keywords- ASP.NET Core MVC, Object-Oriented Programming, Entity Framework Core, PostgreSQL, Cyber Security, Rule-Based Analysis.

I. INTRODUCTION

Understanding suspicious software behavior in cyber security does not require building offensive tools. In an educational setting, a safer approach is to analyze sample indicators and classify their risk without implementing malicious functionality. This project follows that approach by providing a web-based system that evaluates manually entered process properties related to possible keylogger-like behavior.

The main goal of the project is to demonstrate object-oriented programming concepts such as classes, objects, encapsulation, interfaces, services, and layered architecture in a practical web application. The system also clearly states its ethical boundaries: it does not record keystrokes, does not use keyboard hooks, does not monitor live processes, and does not generate harmful code.

II. PROBLEM DEFINITION AND OBJECTIVES

Determining whether a process is risky requires evaluating multiple indicators such as publisher information, file location, startup behavior, background execution, network activity, and keyboard access suspicion. Interpreting these indicators separately can be inconsistent and error-prone. The project solves this problem through a centralized rule base and a simple scoring mechanism.

The project objectives are:

- Store manually entered process information.
- Generate a risk score using configurable detection rules.
- Classify each analysis as low, medium, or high risk.

- Display triggered rules and recommendations in the analysis details.
- Provide rule and user management for administrator users.
- Make ethical usage limits visible to the user.

III. TECHNOLOGIES USED

The project is an ASP.NET Core MVC application targeting .NET 8. Entity Framework Core is used for data access, PostgreSQL is used as the relational database, and the Npgsql provider connects the application to PostgreSQL. ASP.NET Core Cookie Authentication handles login and session management. The user interface is built with Razor Views and Bootstrap. Docker Compose is included to run the PostgreSQL service in a standard local environment.

Table I. Main Technologies

Technology	Role in the Project
ASP.NET Core MVC	Web application structure based on controllers, views, and models
Entity Framework Core	ORM, migrations, relationships, and seed data
PostgreSQL	Storage for users, analyses, rules, recommendations, and logs
Cookie Authentication	Login, logout, and role-based access control
Bootstrap	Responsive and readable user interface
Docker Compose	Standardized PostgreSQL startup for development

IV. SYSTEM ARCHITECTURE

The application uses a layered MVC architecture. Controllers receive HTTP requests, call business logic through service interfaces, and return Razor views with suitable models or view models. Database operations are not placed directly in controllers; they are handled by service classes. This design improves readability and separates responsibilities.

Table II. Layers and Responsibilities

Layer	Example Files	Responsibility
Models	User, ProcessReport, DetectionRule	Entities and relationships
Data	ApplicationDbContext	DbSet definitions, relationships, enum conversions, seed data
Services	AnalysisService, AuthService	Business rules and data access flow
Controllers	AnalysisController, UsersController	HTTP flow and routing
ViewModels	ProcessReportCreateViewModel	Form and screen-specific models
Views	Razor .cshtml files	User interface

V. DATABASE DESIGN

The database design uses a relational structure that makes the analysis process traceable and reportable. The User table stores user information, the Role table stores authorization roles, and the ProcessReport table stores manually entered process records. Each process report has one AnalysisResult. Every triggered rule is stored in TriggeredRule. The Recommendation table provides guidance according to risk level, while the AuditLog table records user actions such as login, logout, analysis creation, and administrative changes.

ApplicationDbContext defines a unique index for user e-mail addresses. It also defines a one-to-one relationship between ProcessReport and AnalysisResult. The DetectionRuleType and RiskLevel enums are stored as strings in the database. Seed data creates Admin and Student roles, a default admin user, default detection rules, recommendations, and a sample high-risk analysis.

VI. OBJECT-ORIENTED PROGRAMMING APPROACH

Object-oriented programming principles are applied directly in the project. Entity classes represent data and relationships, while service classes represent behavior and

business rules. This distinction supports encapsulation and separation of concerns.

A. Encapsulation

The details of risk calculation are encapsulated inside AnalysisService. The controller only calls the service and does not need to know how scoring, triggered rule creation, or risk classification works internally.

B. Abstraction

Interfaces such as IAnalysisService, IAuthService, IUserService, and IDetectionRuleService decouple controllers from concrete service classes. This makes the business layer easier to maintain and replace.

C. Polymorphism and Dependency Injection

Services are registered in the dependency injection container using AddScoped. Controllers depend on interface types, which supports polymorphism because different implementations can be provided for the same interface if needed.

D. Enum Usage

DetectionRuleType and RiskLevel represent fixed behaviors and classifications in a type-safe way. This reduces errors that could occur with plain string comparisons.

VII. ANALYSIS ALGORITHM

The analysis process starts by comparing user-entered process properties with active detection rules. Each rule represents a suspicious behavior indicator. If a rule is triggered, its risk point is added to the total score and the reason is saved. The total score is capped at 100.

Total Risk = min(100, sum of triggered rule points)

The risk level is classified as low between 0 and 30, medium between 31 and 60, and high between 61 and 100. These thresholds are documented in the README file and implemented in the CalculateRiskLevel method.

Table III. Default Detection Rules

Rule	Behavior Indicator	Point
Unknown Publisher	Publisher value is empty, unknown, or bilinmeyan	15
Runs at Startup	Automatically runs during system startup	15
Temp/AppData Location	Runs from a Temp or AppData directory	20
Keyboard Access Suspicion	Manual keyboard access suspicion flag	25
Suspicious Network Connection	Network connection is reported	15
Fake System File Name	Uses a name that imitates a system file	20
Runs in Background	Background execution behavior	10

VIII. APPLICATION MODULES

The Login module validates the user e-mail and password, then starts a cookie-based session. The Dashboard module displays the total number of analyses, counts by risk level, and recent reports. The New Analysis module provides a form with fields such as process name, file path, publisher, network connection, startup behavior, and suspicious flags. The Analysis Details module displays triggered rules, total score, risk level, and recommendations.

Administrator users can add, update, activate, or deactivate rules through the Detection Rules Management page. The User Management page supports creating users and changing user activity status. The Reports page shows total analyses, risk distribution, and the most frequently triggered rules. The Ethical Usage page clearly separates what the system does and what it does not do.

IX. SECURITY AND ETHICAL BOUNDARIES

This project does not write keyloggers, does not record keystrokes, does not monitor live processes, and does not generate malicious code. It only analyzes sample data for educational and defensive purposes.

The application uses [Authorize] attributes for authorization. Critical areas such as detection rule management and user management are restricted to the

Admin role. Form submissions use `ValidateAntiForgeryToken` to provide basic protection against CSRF. Passwords are not stored as plain text; the project generates a SHA-256 hash with a project-level salt. User actions are written to the `AuditLog` table to improve traceability.

X. VALIDATION AND SAMPLE SCENARIO

The seed data contains a suspicious sample record named `svhost.exe`. Its file path is under AppData, the publisher is Unknown, and indicators such as startup execution, network connection, keyboard access suspicion, background execution, and fake system file name are enabled. Because this record triggers all default rules, the total score is capped at 100 and the risk level becomes high.

Form validation is implemented with data annotations in the view model classes. Required fields and maximum string lengths are validated before the analysis is saved, which helps prevent incomplete or invalid user input.

XI. CONCLUSION

The Keylogger Detection System is a defensive and ethically bounded web application developed for an Object-Oriented Programming course. It combines ASP.NET Core MVC, service-based business logic, Entity Framework Core relational modeling, and role-based user management. The rule-based analysis algorithm is simple, understandable, and extensible. In the future, the project can be improved by adding new detection rule types, configurable thresholds, advanced charts, export features, or additional recommendation logic.

REFERENCES

- [1] Microsoft, "ASP.NET Core MVC documentation," Microsoft Learn.
- [2] Microsoft, "Entity Framework Core documentation," Microsoft Learn.
- [3] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [4] OWASP Foundation, "Secure Coding Practices and Web Application Security Guidance."
- [5] Keylogger Detection System project source code and README file.